DD-RTREE: A dynamic distributed data structure for efficient data distribution among cluster nodes for spatial data mining algorithms

Jagat Sesh Challa, Poonam Goyal, Nikhil S., Aditya Mangla, Sundar S. Balasubramaniam, Navneet Goyal
Advanced Data Analytics & Parallel Technologies Laboratory
Department of Computer Science & Information Systems
Birla Institute of Technology & Science, Pilani, Pilani Campus, India
{jagatsesh, poonam, h2014105, f2012209, sundarb, goel}@pilani.bits-pilani.ac.in

***Abstract*— Parallelizing data mining algorithms has become a necessity as we try to mine ever increasing volumes of data. Spatial data mining algorithms like DBSCAN, OPTICS, SLINK, etc. have been parallelized to exploit a cluster infrastructure. The efficiency achieved by existing algorithms can be attributed to spatial locality preservation using spatial indexing structures like k-d-tree, quad-tree, grid files, etc. for distributing data among cluster nodes. However, these indexing structures are static in nature, i.e., they need to scan the entire dataset to determine the partitioning coordinates. This results in high data distribution cost when the data size is large. In this paper, we propose a dynamic distributed data structure, DD-RTREE, which preserves spatial locality while distributing data across compute nodes in a shared nothing environment. Moreover, DD-RTREE is dynamic, i.e., it can be constructed incrementally making it useful for handling big data. We compare the quality of data distribution achieved by DD-RTREE with one of the recent distributed indexing structure, SD-RTREE. We also compare the efficiency of queries supported by these indexing structures along with the overall efficiency of DBSCAN algorithm. Our experimental results show that DD-RTREE achieves better data distribution and thereby resulting in improved overall efficiency.**

Keywords- Data mining, data distribution, spatial locality, neighborhood queries, k-NN queries, density based clustering.

I. INTRODUCTION

Big Data Analytics is becoming more and more popular trying to make sense of humongous volumes of data that is getting generated these days. Data Mining constitutes an important part of Big Data Analytics. Many researchers have parallelized data mining algorithms to leverage a cluster infrastructure. In many data mining algorithms, it helps to distribute data in such a way that spatial locality is preserved. By preservation of spatial locality, we mean that for a given data point p, the data points surrounding p are available locally in the same compute node. This reduces inter-node communication required for the execution of the algorithm. Algorithms that exploit spatial locality include: density based clustering algorithms (DBSCAN[1], OPTICS[2], etc.) and hierarchical clustering algorithms (SLINK [3], CLINK[4], etc.). Some recent attempts to parallelize these algorithms over a cluster are presented in [5]–[8]. The above algorithms follow a typical pattern in execution. The execution steps can be broadly categorized into four steps as illustrated in Fig 1. In the step 1, the data is distributed across the compute nodes using a suitable distribution method. In step 2, every compute node (or machine) retrieves data points that are required for its local computations from other nodes. In step 3, local computations are performed at each compute node independently. In step 4, the results of all local computations are reduced to get the global output. The data distribution step plays a pivotal role in optimizing the execution of the other steps. For example, in step 3, some data mining algorithms may require to execute neighborhood and k-nearest neighbor (k-NN) queries. The execution of such queries for a data point p, becomes efficient when the data required by these queries is available locally. If this requirement is not met, then we need to retrieve data from other compute nodes thereby incurring inter-node communication cost. This is illustrated in Fig. 2 in which three different data distributions are shown over four nodes. – A, B, C and D. If an ε-neighborhood query (See Fig. 4) is to be executed on all points of a given compute node, all the other nodes needs to be accessed and lot of points have to be retrieved from them, in the case shown in Fig. 2(a). A lesser number points need to be retrieved in case of Fig. 2(b), and still fewer points in case of Fig. 2(c). This is because spatial locality is best preserved in Fig. 2c. Thus, a good spatial distribution helps in reducing inter-node communication and computations in merging (step 4), making parallel data mining algorithms, efficient.

Researchers have used various spatial indexing structures like k-d-tree [5]–[7] and *grid* files [9] for distributing data. Although, these indexing structures achieve good spatial locality, the distribution is *static*, i.e., it is required to scan the entire data to do the partitioning. It would thus become very expensive to use them for distributing large datasets. This is because, memory in a single machine may not be sufficient to accommodate the entire dataset, and thus leading to a lot of disk I/O while computing the partitioning coordinates. To the best of our knowledge, in only one instance, k-d-tree like distribution has been done in a very naïve manner for distributing large data using sampling [8]. This technique is very expensive, and has huge communication overheads.

There are a few dynamic variants of distributed data structures proposed in literature which include - *Parallel R-tree* [10], [11], Distributed B-link tree [12], Distributed Random tree [13], *Master-Client R-tree* [14], *Upgraded Parallel R-tree* [15], SD-RTREE [16], etc. Most of these data structures focus on increasing the degree of parallelism to get optimal query performance. SD-RTREE is the most recently proposed distributed data structure [16], which reduces the

978-1-4673-9005-7/16/$31.00 ©2016 IEEE

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

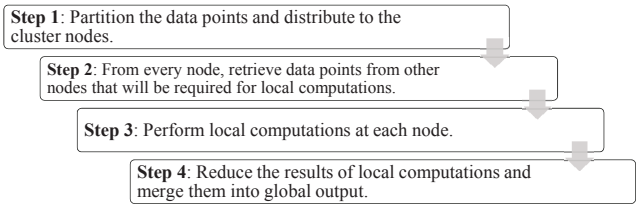


Figure 1. Dataflow of a typical parallel data mining algorithm

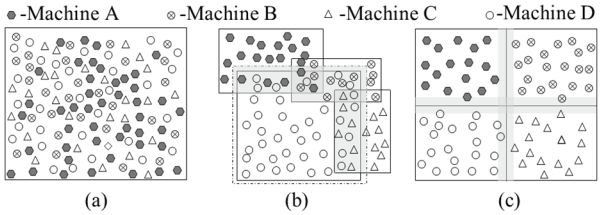


Figure 2. Spatial locality preservation (a) None (b) Moderate (c) High

communication overheads, in construction and querying, using AVL Tree [17] and R-tree [18] spatial containment principles. It uses R-tree's minimum bounding rectangles (MBRs) in its structure, along with its functionality of node (R-tree node) splits. It supports dynamic insertions and shows good scalability in terms of adding compute nodes to the cluster. However, SD-RTREE doesn't preserve spatial locality satisfactorily since its re-distribution strategy is based on k-NN search (See Section 2). Also, it doesn't ensure good load balancing (See Section 4A).

Thus, there is a need for an efficient dynamic distributed data structure that will preserve spatial locality satisfactorily and ensures load balancing, thereby improving query performance in data mining algorithms.

The main contribution of this paper is DD-RTREE, a novel dynamic distributed data structure, based on R-tree [18]. It preserves spatial locality and ensures proper load balancing. The design of DD-RTREE (see section 3) also makes it dynamic, i.e., data can be added incrementally, and compute nodes can also be added incrementally, if required. DD-RTREE structure consists of R-trees at two levels. The first level R-tree is the *index-R-tree* (IR-TREE), which serves as the index during construction/insertion. The second level comprises of multiple R-trees for each compute node (*MR-TREE*), which indexes data belonging to that node (See section 3 for details).

We evaluate our proposed data structure using four parameters – (1) spatial locality, (2) communication cost (3) construction & querying time, and (4) performance of data mining algorithms. For the above parameters, we compare DD-RTREE with SD-RTREE and also with the case when data is distributed randomly. We use SD-RTREE for our comparative study as this is the best suitable existing distributed data structure that can be used for data distribution. DD-RTREE is found to outperform SD-RTREE. We have also used DD-RTREE to optimize k-d-tree like data distribution given in [8], to obtain completely disjoint partitions and optimal load balancing.

The rest of the paper is organized as follows. Section 2 gives brief literature survey on multi-dimensional indexing structures and their parallel versions. It also gives an overview of R-tree and SD-RTREE data structures. Section 3 presents the DD-RTREE structure along with its operations and complexity analysis. Section 4 presents the quality and performance evaluation. Section 5 concludes our work and gives future directions.

II. RELATED WORK & BACKGROUND

In this section, we give a brief survey of existing literature followed by an overview of R-tree and SD-RTREE.

Literature Survey. Distributed Data Structure (DDS) is a data structure that is used in a message passing system (typically a cluster of compute nodes). DDS is composed of a data organization scheme and a set of distributed access protocols to enable compute nodes to issue query and modification instructions and get appropriate responses. The data organization scheme acts like an index to the collection of local data structures that are stored at each compute node [19]. Many DDSs are proposed in literature for various domains including peer-peer network overlays, data analytics, social network mining, etc. [13], [19]–[21]. These data structures are typically used to index data for efficient query processing, routing, etc. Distributed versions of R-tree and its variants proposed in literature include - *Parallel R-tree* [10], [11], Distributed B-link tree [12], Distributed Random tree [13], *Master-Client R-tree* [14], *Upgraded Parallel R-tree* [15] and SD-RTREE [16], which were originally proposed for database systems to improve the efficiency of various queries. Most of these focus on optimizing communication overheads and increasing degree of parallelism to get optimal query performance [10]–[12]. They achieve this by organizing data in such a way that, multiple compute nodes can be simultaneously accessed to answer a query. Hence, they are not suitable for data distribution. The most recent variant is the SD-RTREE [16], which is a hybrid structure based on *AVL tree* [17] and *R-tree*. It supports dynamic insertions and shows good scalability. But it doesn't guarantee good load balancing (See Section 4A). Also, the re-distribution strategy it uses is based on k-NN search, which does not guarantee to preserve good spatial locality. As is evident from above discussion, none of the data structures target preservation of spatial locality in their distribution and at the same time achieve good load balance and give optimal query performance for parallel data mining algorithms.

A few multi-dimensional indexing structures used in literature for data distribution include - *k-d-tree* [5]–[7] and *grid* files [9]. They are used for distributing datasets across compute nodes in a cluster to run parallel algorithms. A major disadvantage of these data structures is that they are static, i.e. they need to read entire dataset before they start partitioning, and thus are costly for indexing large data. Patwary et al. [8] proposed a naïve k-d-tree like distribution for distributing large data using sampling. However, this technique is very expensive and has huge communication overhead. Also, the partitioning scheme is static here as well.

Apart from tree based data structures, there is an approach proposed to maintain spatial locality that is based on Vornoi diagrams [22]. However, this scheme is also static and needs to scan the entire dataset prior to its construction.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

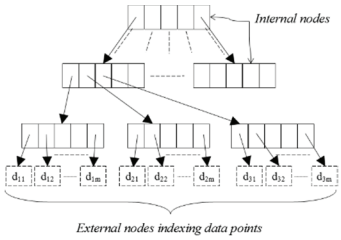
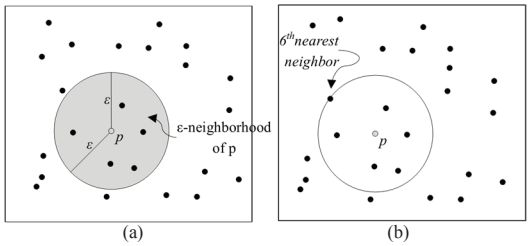


Figure 3. R-tree structure



(a) (b)
Figure 4. (a) ε-neighborhood query (b) k-NN query for k=6

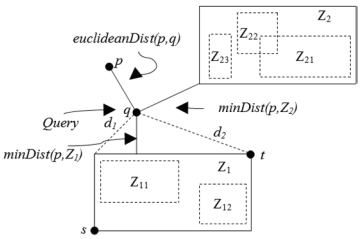


Figure 5. Illust. various distance measures

R-tree. *R-tree* [18] is a commonly used multi-dimensional indexing structure. Fig. 5 illustrates the structure of an R-tree: it consists of two kinds of tree nodes – internal nodes and external nodes. Internal nodes store d-dimensional minimum bounding rectangles (MBRs) which bounds all the data points indexed at the subtree rooted at that node. External nodes consist of entries that index d-dimensional data points. Each node (both internal and external) has a minimum of m entries and a maximum of M entries stored in it and generally $m \leq M/2$. R-tree is constructed by incremental insertions of a list of data points. Insertion of a data point happens in a top-down recursive fashion into the sub-tree rooted at a given node beginning from the root. The average complexity of an insertion into R-tree is $O(\log_m N)$, and thus that of construction is $O(N \log_m N)$. Here, N is the data size. The deletion in R-tree happens similar to insertion [18].

R-tree supports neighborhood and k-NN queries in logarithmic time on an average. An ε-neighborhood query returns all the data points lying within an ε distance from query point p as explained in Fig. 4a. The k-NN query is a query that returns the k closest data points to a given query point p as illustrated in Fig. 4b for k=6. The best known algorithm for k-NN search over R-tree is the BF-kNN [23]. It is a greedy algorithm & uses a min-priority queue based on *min distance* measure. *Min distance* is the minimum possible distance from a given point p to any other point lying in an MBR Z as illustrated in Fig. 5.

SD-RTREE. SD-RTREE [16] is a distributed data structure that works over a cluster of compute nodes. Its structure is conceptually similar to that of a classical AVL tree [24], with its data organization principles borrowed from the R-tree spatial containment relationship. Fig. 6 illustrates its design. It is a height balanced binary tree (AVL tree), mapped to a set of servers, satisfying the following properties:

- Each internal node (called *routing node*), has exactly two children. Each internal node maintains left and right *directory rectangles* (dr) which correspond to MBRs of the left and right subtrees respectively.
- Each leaf node (called *data node*), stores an index of data points stored at that machine.

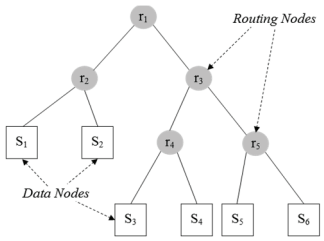


Figure 6. SD-RTREE Structure

SD-RTREE has S leaves and S-1 internal nodes which are distributed among S machines. Each machine in the cluster stores a pair (rn_i, dn_i), rn_i being a *routing node* and dn_i a *data node*. Each rn_i stores its height; dr; two links pointing to its left and right children; its parent id; and the overlapping coverage (OC). OC is an array that contains parts of its dr that are overlapping with other machines. OCs are used for execution of queries, and are updated when insertions or deletions happen. Each data node indexes the portion of the dataset allocated it along with its dr; id of its parent routing node; and the OC. The data stored at data nodes is essentially indexed in a local R-tree. Every data node has max capacity that is, max no. of data points it can index.

SD-RTREE also maintains an image I of the distributed tree which is accessed by the application client before triggering an insertion or a query. The image stores the meta-data of all routing nodes as *routing links* and data nodes as *data links*. The image helps in redirection of an insertion or query message to the appropriate machine in the cluster. The image generally resides in a server from which an application calls all its operations. It is maintained/updated by image adjustment messages (IAM) from the machines that are affected by any insertion or deletion of data points.

Construction of SD-RTREE: SD-RTREE construction happens by repeated insertions into it. The pseudo codes of the functions used for insertion are given in Algorithms 1, 2 & 3. As a first step in insertion of data point p, *we search its image I as follows:*

- All the data links are searched first; a link whose dr contains p is kept as a candidate; when several candidates are found, the link with smallest dr is chosen.
- If no data link has been found, the list of routing links are considered in turn; among the links whose dr contains p, if any, one chooses those with minimal height; if there are several candidates, the link with smallest dr is chosen.

If a data link is chosen in the above step, then we send a message to execute INSERT-IN-LEAF (Algorithm 1) to insert p into that data node dn. In this function, we first check if dn is full or not. If dn is not full, we simply insert p in its repository (Repositories are implemented as R-trees). Else if dn is full and there is a new machine available in the cluster, we first insert p into dn and then, a split is performed similar to the R-tree split and half of the entries are shifted to the new machine. The new machine is added to the AVL tree index. This addition could lead to violation of AVL tree's height balance property. So, a few rotations similar to AVL tree are performed to restore it. All this is handled by SPLIT-AND-ADJUST function. Then an image adjustment message (IAM) is sent to the master informing the split. On the other hand, if

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

there is no machine available in the cluster, i.e. all machines are already allocated, then REDISTRIBUTE procedure is called which performs re-distribution of points to create space in dn and then p is inserted into it. In re-distribution, space is created for p in $S(= dn)$, by trying to shift one of its data points to some other machine. First, its closest ancestor S_p is found in the AVL tree index, which has one of its child as non-full. This would involve additional MPI messages to the master. Without loss of generality we can assume that S_p's right child S_r is non full. Then we determine the centroid P of the dr of S_r. Now, with p as the query point, we execute 1-NN search (explained in next subsection) over the subtree rooted at S_l which is the left child of S_p that was full. The machine S actually lies in the subtree rooted as S_l. Then, the resultant nearest neighbor q is inserted into the subtree rooted at S_r. This point q might have been picked up from any node in the subtree rooted as S_l and it may not have been from S. So, the entire process is repeated until a data point is picked from S and is re-distributed to some other machine. When space is created in S, p is inserted into it.

If instead a routing link is chosen in the first step, then INSERT-IN-SUBTREE (Algorithm 3) is called. It inserts p into the subtree rooted at that rn in a top down recursive fashion using expansion area principles similar to R-tree.

The deletion in SD-RTREE happens in a similar way as that of insertion. It could result in node underflows leading to removal of machines from the SD-RTREE index. Since, deletion is not very much relevant in our context of data distribution, we don't discuss it further.

Queries supported by SD-RTREE. SD-RTREE typically supports two queries: region query and k-NN query. The server hosting the image (I) issues MPI messages for query execution to the compute nodes in the cluster and receives a few messages as the output of the queries. Due to limited space we do not explain them. Also, the execution pattern of above queries in terms of flow of MPI messages is almost the same as that of DD-RTREE (explained in next section). Hence, they become comparable.

We implement the IMCLIENT variant of SD-RTREE [16], where we have the image stored in a service providing server or the master. This is the most suitable for data distribution as we assume that the dataset is initially stored on this master.

The re-distribution in SD-RTREE, creates space for only one data point in the machine that has overflown. If we have to insert another data point in the same machine, we need to do re-distribution again. Also, each re-distribution triggers many MPI messages and thus, repeated re-distributions could become a bottleneck in its construction time. In our proposed DD-RTREE, we use buffering technique to do bulk insertions rather than point by point insertion. Also, the re-distribution strategy shifts points in bulk to other compute nodes, and thus save a lot of MPI messages (See Section 4).

ALGORITHM 1. INSERT-IN-LEAF

```
procedure INSERT-IN-LEAF(p, dn)
    if (dn's repository is non full) then
        insert p into dn's data repository
        update dn's OC
    else if (dn's repository is full) then
        if (more machines are available) then
            insert p into dn's data repository
            update dn's OC
            SPLIT-AND-ADJUST(dn)
            Send an IAM message to Master
        else
            REDISTRIBUTE(d)
            insert p into d's data repository
            update d's OC
```

ALGORITHM 2. INSERT-IN-SUBTREE

```
procedure INSERT-IN-SUBTREE(p, r)
    l = CHOOSESUBTREE(r)
    if (l is data node) then
        send a message to l to perform INSERT-IN-LEAF(p, l)
    if (l is a routing node) then
        update OC of l's outer node
        send a message to l to perform INSERT-IN-SUBTREE(p, l)
```

ALGORITHM 3. REDISTRIBUTE-SD-RTREE.

```
procedure REDISTRIBUTE(S)
    while (S is full) do
        Find the closest ancestor S_p of S such that S_p has one of its child as non-
        full; without loss of generality say S_r = S_p.rightChild is not full.
        Determine P the centroid of the dr of S_r
        q ← Find 1st nearest neighbor of P in the subtree rooted at S_l where S_l is the
        left child of S_p and S_l is full
        Insert- In- Subtree(q, S_r) and remove q from its node.
```

III. DD-RTREE

DD-RTREE is a dynamic distributed data structure that resides on a cluster of compute nodes. DD-RTREE is designed to distribute data across multiple compute nodes with the following objectives: maximizing spatial locality; achieving good load balance; minimizing inter-node communication for its construction, and minimizing execution time of queries and spatial data mining algorithms. DD-RTREE is first distributed spatial indexing structure which tries to achieve the above objectives. The design of DD-RTREE also makes it dynamic, i.e., data can be added incrementally and compute nodes can also be added incrementally, if required.

A. DD-RTREE Design

DD-RTREE comprises of R-trees at both the levels. The first level R-tree is the *index-R-tree* (IR-TREE), which serves as the index to the entire structure and resides in a master compute node or a server from where all the instructions are issued. The second level comprises of multiple R-trees stored one each at each machine of the cluster (MR-TREE). MR-TREE indexes data points that belong to its machine. IR-TREE satisfies the following properties:

- Each node of IR-TREE has a minimum of Im and maximum of IM entries indexed in it, except the root which can have less than Im entries.
- Each internal node consists of MBRs which store the bounding information of all the objects indexed at their respective sub-trees.
- Each external node stores MBR information of all the points indexed in a machine. In other words, it stores the MBR of the root of an MR-TREE. It also stores the $machineID$ of the machine where that MR-TREE is stored and a count of points (cnt) indexed in it.
- Each external node also contains a buffer of a fixed capacity bc, that temporarily stores data points before pushing them into the corresponding MR-TREE.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

MR-TREES are the R-trees with the native R-tree properties. Each machine has a capacity mc which is the maximum number of data points it can index.

B. DD-RTREE Construction

The pseudo codes of algorithms for construction of DD-RTREE are explained in Algorithms 4, 5, 6 & 7. Initially an empty IR-TREE is created. Then data points in the data list DL are inserted into the DD-RTREE one after the other. In insertion, we first find the most appropriate leaf ($ILeaf$) of the IR-TREE to insert a data point p, by the usual R-tree recursive top down search using expansion area principles. We then store p in $ILeaf.buffer$ and update the MBRs of IRTREE in a bottom up manner similar to that of R-tree. We also increment $ILeaf.cnt$, which indicates the number of points stored or to be stored in the corresponding machine. If $ILeaf.buffer$ reaches buffer capacity (bc), then the points indexed in the buffer are flushed into the corresponding machine, by inserting them into its MR-TREE. If at this point, the machine exceeds its capacity, it tries to identify if there is any other new machine in the cluster available by contacting the master. If, yes, the machine splits itself into two equal halves by the usual R-tree split algorithm and one of the halves is transmigrated to the new machine and two new MR-TREES are created. This would lead to MBR updates in the MR-TREES as well as the IRTREE, which are done using a few MPI messages. If there is no free machine available in the cluster, then the machine performs re-distribution. In this process, we try to shift a few points from the current machine to few other machines so that some space is created for incoming data points. Re-distribution is explained in the next subsection. Finally, after all insertions finish, all the buffers of IR-TREE are flushed into their respective MR-TREES.

ALGORITHM 4. DD-RTREE Construction. Input: List of data points $\boldsymbol{DL}$. Output : IR-TREE $\boldsymbol{ITree}$ of the DD-RTREE constructed.

```
procedure CONSTRUCT-DD-RTREE(DL)
    Initialize an empty IR-TREE ITree
    for each point in DL do
        INSERT-IN-DD-RTREE(p, ITree)
    for each leaf in ITree do
        FLUSH-BUFFER(leaf.buff)
    return ITree
```

ALGORITHM 5. INSERT-IN-DD-RTREE

```
procedure INSERT-IN-DD-RTREE(p, ITree)
    ILeaf = CHOOSE-LEAF(p, ITree)
    Insert p into ILeaf.buffer
    Update MBRs of ITree in a bottom up manner
    Increment ILeaf.cnt by 1
    if (ILeaf.buffer is FULL) then
        Send FLUSH-BUFFER message to machine with ID- ILeaf.machineID to
        empty the contents of ILeaf.buffer into its MR-TREE
```

ALGORITHM 6. FLUSH-BUFFER.

```
procedure FLUSH-BUFFER(buff)
    for each point p in buff do
        INSERT-INTO-R-TREE(p)
    if (no. of points in this machine exceed mc) then
        if (there exists an empty machine in the cluster) then
            SPLIT-AND-ADJUST()
        else
            REDISTRIBUTE()
```

ALGORITHM 7. REDISTRIBUTE-DD-RTREE.

```
procedure REDISTRIBUTE(S)
    Compute the proportion of points to be shifted to each overlapping node.
    If some node in them is full, recursively call REDISTRIBUTE over them to create
    space in them and then shift points
    If (sufficient points are not shifted) then
      Shift based on k-NN
```

Re-distribution. Unlike SD-RTREE, where only one point is shifted out from a full compute node, we shift points in bulk, i.e. we shift multiple points in one re-distribution, creating more space for incoming points. This helps in reducing the communication overheads for subsequent insertions. The algorithm is as follows: when a compute node A is full, we first identify if there are any other compute nodes (B or C or both) whose MBRs are overlapping with that of A. If yes, we try to shift a maximum of τ points in total from the overlapping regions from A to their respective machines B or C. In practice $\tau = x * bc$ where $x \in [0,1]$. If any of B or C is full, then we first recursively apply re-distribution over that compute node to create space in it and then shift points from A into it. If however, we don't have sufficient space in the overlapping machines to shift τ points, or there are no overlapping compute nodes, we try to shift them to non-full machines that are not overlapping with A based on k-NN. In this, we compute min-distances from the centroid of these non-full overlapping compute nodes to A and order them in increasing order of min distance. Then depending on space availability in each of these machines, we greedily transfer points to them. For example if we have to shift points to node B having remaining space y, then we trigger y-NN using centroid of B over points in A and shift those y points from A to B. Similarly we shift points to other non-overlapping machines. In practice, the k-NN based re-distribution is triggered very less number of times. So, it's the overlap based re-distribution strategy that suffices and ensures that spatial locality is not affected.

DD-RTREE advantages. We can see from the above discussion, that the design of DD-RTREE achieves minimal overlap among the bounding rectangles of the machines. This is because, all the algorithms governing construction of DD-RTREE are based on R-tree's construction principles. DD-RTREE exhibits good spatial locality and efficient query performance (verified by experiments in next section). Buffers attached to the leaves of IR-TREE enable reduction in communication overheads during the construction phase. Although the redistribution strategy of DD-RTREE involves high communication overhead, it is expected to be more efficient because the number of redistributions occurring in total is quite less when compared to that of SD-RTREE. As a result, DD-RTREE has a lesser construction time. This has been verified by experiments (see next section). Also the redistribution strategy is based on the principles of minimizing overlap among the bounding rectangles of the machines when compared to that of SD-RTREE which is based on k-NN only. Thus, it gives better locality in distribution and efficient query performance. DD-RTREE serves as an efficient data distribution method to distribute data across compute nodes in a cluster and thereby improving the efficiency of the parallel spatial data mining algorithms.

C. Queries supported by DD-RTREE

DD-RTREE supports ε-neighborhood queries and k-NN queries. Queries are issued from the master or the server where the IR-TREE is stored.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

Neighborhood Queries. The pseudo code explaining the execution of ε-neighborhood query over DD-RTREE is given in Algorithm 8. We first construct an ε-extended region r by extending the coordinates of p in both directions across all dimensions. We then perform a region query over IR-TREE ($Itree$) similar to the top-down recursive search in an R-tree, to retrieve all the machines that overlap with r. Then for each of the leaves retrieved, we pass an MPI message (FORWARD-NBH-QUERY) asking it to perform neighborhood query over its MR-TREE using r. The results of all of them are collected back at the master and are collectively returned. The number of MPI messages required to perform this query is double the number of machines visited. We can also minimize the messages by doing a sequential visit of all those leaves overlapping with r, reducing MPI messages to no. of machines visited +1. But the first approach works faster for big datasets as it works in parallel.

Nearest Neighbor Queries. The k-NN query (Algorithm 9) uses one max priority queue $NbhPQ$ of size k to store k nearest neighbors. It also uses two min priority queues - PQ_{mmd} and PQ_{md}, into which all the *machineIDs* are inserted with their *minMaxdist* and *mindist* from p as keys respectively. We then find machine S that could contain p from $ITree$, by doing a top-down recursive search over it similar to R-tree. An MPI call is then made to S to execute FORWARD-KNN-QUERY and send all three priority queues to S. In the function FORWARD-KNN-QUERY, being executed at S, we first remove S from both PQ_{mmd} and PQ_{md} and then perform a local k-NN search over MR-TREE of S. If S is the first machine of the cluster we are visiting, then we add all the k nearest neighbors to $NbhPQ$. Else we insert only those neighbors that are at a distance less than the distance between p and the current k^{th} nearest neighbor ($tempDist$) and the $NbhPQ$ is then updated. After this, we do a removeMin() operation on both PQ_{mmd} and PQ_{md} and store the retrieved (*machineID, distance*) pairs in $(id1, mmd)$ and $(id2, md)$ respectively. mmd is the distance at which there is at least one point in machine with *machineID* - $id1$ (S_{id1}). md is the minimum possible distance between p and any point in the machine with *machineID* - $id2$ (S_{id2}). If $mmd < tempDist$, then there is at least one data point in S_{id1} which is at a distance less than $tempDist$ from p. So, we forward the search request to S_{id1}. If not, we check if $md < tempDist$, then we forward the request to S_{id2}. If md would have been greater than $tempDist$, we don't explore this machine. Now, if both the above criteria fail, we don't need to visit any more machines and we simply return the result. The number of MPI calls required for execution of this query is equal to number of machines visited +1.

ALGORITHM 8. DDR-NBHQUERY. Input: query point $\boldsymbol{p}$, epsilon value $\boldsymbol{\varepsilon}$, IR-TREE $\boldsymbol{ITree}$ Output: All the points in the $\boldsymbol{\varepsilon}$ neighborhood of $\boldsymbol{p}$.

1: **procedure** DDR-NBHQUERY($p, \varepsilon, ITree$)
2: construct an ε-extended region r of p
3: perform a top down recursive search over $ITree$ to find the leaves of $ITree$ that overlap with r and store them in a Queue, MQ
4: **for each** $leaf$ in MQ **do**
5: FORWARD-NBH-QUERY(p, ε) to MR-TREE of $leaf$ and collect the results

6: **procedure** FORWARD-NBH-QUERY(p, ε)
7: R-NBHQUERY($p, \varepsilon, this.MRtree, tempList$) // accumulates ε-neighborhood of point p lying in the MR-TREE of the current machine to $tempList$.
8: return $tempList$ to the master

ALGORITHM 9. DDR-K-NNQUERY. Input: query point $\boldsymbol{p}$, $\boldsymbol{k}$, a max-priority queue $\boldsymbol{NbhPQ}$ of size $\boldsymbol{k}$, IR-TREE $\boldsymbol{ITree}$, Output: $\boldsymbol{k}$ nearest neighbors stored in $\boldsymbol{NbhPQ}$.

1: **procedure** DDR-K-NNQUERY($p, k, NbhPQ, ITree$)
2: Create two min-priority queues PQ_{mmd} and PQ_{md}.
3: **for each** machine $i \in ITree.leaves$ **do**
4: insert $machineID_i$ into PQ_{mmd} with *minMaxdist* and into PQ_{md} with *mindist* from p as keys
5: Find the machine S that contains p from $ITree$
6: FORWARD-KNN-QUERY ($S, p, NbhPQ, PQ_{mmd}, PQ_{md}$) // makes an MPI call to machine S.

7: **procedure** FORWARD-KNN-QUERY ($S, p, NbhPQ, PQ_{mmd}, PQ_{md}$)
8: Remove S from PQ_{mmd} and PQ_{md}
9: Perform locally the k-NN search over MR-TREE of S
10: **if** ($NbhPQ$ is empty) **then**
11: Insert all the k nearest neighbors in $NbhPQ$ with distance form p as keys.
12: **else**
13: $tempDist \leftarrow$ distance between p and k^{th} nearest neighbor from $NbhPQ$
14: Insert only those neighbors that are at a distance < $tempdist$ from p and update $NbhPQ$
15: **if** (PQ_{mmd} is empty **OR** PQ_{md} is empty) **then**
16: **return** NbhPQ
17: $(id1, mmd) \leftarrow$ removeMin(PQ_{mmd}); $(id2, md) \leftarrow$ removeMin (PQ_{md})
18: **if** ($mmd < tempDist$) **then**
19: FORWARD-KNN-QUERY ($S_{id1}, p, NbhPQ, PQ_{mmd}, PQ_{md}$) // continuing search on machine S_{id}
20: **else if** ($md < tempDist$) **then**
21: FORWARD-KNN-QUERY ($S_{id2}, p, NbhPQ, PQ_{mmd}, PQ_{md}$) // continuing search on machine hosting$S_{id'}$
22: **else**
9: **return** $NbhPQ$ // return to master

D. k-d-tree like data distribution using DD-RTREE

DD-RTREE supports efficient k-d-tree like data distribution that creates fully disjoint distribution for very large datasets that don't fit in memory of a single compute node. This has been originally used in [8] for random distribution of points (Algorithm 10). This method in is not optimized and an approximate solution based on sampling. In this method, all the nodes get approximately equal number of data points, achieving good load balancing. This partitioning however makes the distribution static, as it is not capable of handling incremental updates. We apply a similar but optimized algorithm to DD-RTREE to make its distribution fully disjoint and compact. The difference in our optimized version is: 1) in step we distribute data using DD-RTREE; 2) the pair of compute nodes chosen in line 6 of pseudo code (Algorithm 10) is based on R-tree *Split* algorithm rather than random pairing, for the first iteration of the algorithm. In this, we use information from Index-R-tree to split the machines into two equal sets similar to R-tree split and we make pairs, picking one from each of the sets. This ensures that both the partners in each pair are geometrically far apart, thus reducing communication in shifting the points and reduction in overall execution time and communication (See section 4).

ALGORITHM 10. NAÏVE-$\boldsymbol{k}$-D-TREE-PARTITIONING FOR LARGE DATA SETS

1: **procedure** NAÏVE-k-D-TREE-PARTITIONING
2: Randomly distribute all the data points to the machines in the cluster.
3: Randomly select a small set of data points from each machine and broadcast them to all other machines in the cluster.
4: Every machine computes the median from the samples it received.
5: Every machine then partitions data into two sets (halves) with one set on the left side of the median and the second on the right side of the median. The partition is performed perpendicular to one of the dimensions which is chosen based upon the criteria of having largest extent.
6: Then in a pair of two, machines exchange its left and right sets such that one machine gets the entire left half and the other gets entire right half.
7: Now for all the machines that are on the left half, steps 2-6 are repeated recursively. They are also repeated for machines on the right half as well.
8: Thus, this algorithm achieves disjoint partitioning in $log\ n$ iterations, where n is the number of machines.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

IV. Performance Evaluation

We evaluate DD-RTREE with respect to (1) spatial locality, (2) communication cost (3) construction & querying time, and (4) performance of parallel spatial data mining algorithms it supports. We compare it with SD-RTREE and randomly distributed data. The details of the datasets are mentioned in Table 1. The first four datasets are synthetic and the rest are real. SR500M2D & SR10M2D are randomly generated. Data in SN100M2D follow normal distribution. SC100M2D consists of synthetically generated well separated clusters equal to number of machines used for a particular experiment. SFONT1M11D, MPAHALO2.8M9D, MPAGD56M3D, MPAGD16M3D and FOF113M3D datasets are taken from Millennium data repository that contains astronomical data of galaxies in the sky [25]. These datasets are skewed in nature and do not follow any distribution. SBUS6M2D dataset contains samples of GPS traces of buses in Shanghai [26].

TABLE 1. DATASETS USED FOR EXPERIMENTATION

S No	Dataset	Data Size	Dimensionality	Reference
1	SR500M2D	500M	2	-
2	SN100M2D	100M	2	-
3	SC100M2D	100M	2	-
4	SR10M2D	10M	2	-
5	SFONT1M11D	1 M	11	[25]
6	MPAHALO2.8M9D	2.8 M	9	[25]
7	MPAGD56M3D	56 M	3	[25]
8	MPAGD16M3D	16 M	3	[25]
9	FOF113M3D	113 M	3	[25]
10	SBUS6M2D	6 M	2	[26]

All the experiments were conducted on a cluster of 32 compute nodes which are IBM x3250 m4 Servers. Each server has - Intel Xeon (64-bit) processor and 32 GB RAM. All the implementations were done in C with MPI library. In all our experiments, we make the following choices by default until and unless explicitly stated. We choose machine capacity (mc) in such a way that 5% of the total capacity of all the machines remains vacant. We choose bc = 10% of the mc.

A. Quality evaluation

We use various quality measures to evaluate the quality of DD-RTREE distribution. There are no specific measures reported in literature to measure spatial locality. So, we the internal quality measures that evaluate the clustering quality, summarized in [27]. They are based on the $N \times N$ proximity matrix W, that stores all pairwise distances among the N points: $W = \{\delta(x_i, x_j)\}_{i,j=1}^{N}$ where $\delta(x_i, x_j)$ is the euclidean distance between $x_i, x_j \in$ dataset. We assume that we are given a clustering $C = \{C_1, \ldots, C_f\}$, comprising of clusters, with $n_i = |C_i|$. Let $y_i \in \{1,2,\ldots f\}$ denote the cluster label for point x_i. Given any subsets S & R of points, we define $W(S,R)$ as the sum of the weights (distances) on all edges with one vertex in S and the other in R. Then, the sum of all intra-cluster weights or *Cohesion* over all clusters is given as: $W_{in} = \frac{1}{2}\sum_{i=1}^{f} W(C_i, C_i)$ and the sum of all inter-cluster weights or *Separation* is given as: $W_{out} = \sum_{i=1}^{f-1}\sum_{j>i} W(C_i, C_j) = \frac{1}{2}\sum_{i=1}^{f} W(C_i, \overline{C_i})$. The number of distinct intra-cluster edges - N_{in}, and inter-cluster edges - N_{out}, are given as: $N_{in} = \frac{1}{2}\sum_{i=1}^{f} n_i(n_i - 1)$ & $N_{out} = \frac{1}{2}\sum_{i=1}^{f}\sum_{j=1, j\neq i}^{f} n_i . n_j$. Based on these results, various measures are defined and summarized in Table 2. The third column in the table indicates the case when that measure represents better clustering. For e.g. lower value BetaCV indicates good clustering. Due to limited space, we omit their discussion. For more details, refer to [27]. Relative Interconnectivity has been borrowed from [28].

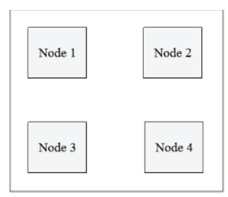


Figure 7. Sample distribution

In order to check the appropriateness of these measures for evaluating quality of spatial locality, we have taken a synthetically generated data set of 10 million data points (two dimensions) containing four well separated clusters as given in Fig. 7. We took this as the base dataset and generated a few more distributions to distort spatial locality by changing the membership of 10%, 20%, 30% randomly chosen data points from their original position to the next best cluster. We also generated a dataset which has the membership assigned

TABLE 2. QUALITY MEASURES

Measure	Formula	When is it better?
BetaCV	$BetaCV = \frac{W_{in}/N_{in}}{W_{out}/N_{out}}$	Low
Normalized Cut (NC)	$NC = \sum_{i=1}^{f} \frac{1}{\frac{W(C_i, C_i)}{W(C_i, \overline{C_i})} + 1}$	High
Modularity (Q)	$Q = \sum_{i=1}^{f}\left(\frac{W(C_i, C_i)}{W(V,V)} - \left(\frac{W(C_i, V)}{W(V,V)}\right)^2\right)$ $where\ W(V,V) = \sum_{i=1}^{f} W(C_i, V)$	Low
Davies–Bouldin Index (DB)	$DB_{ij} = \frac{\sigma_{\mu i} + \sigma_{\mu j}}{\delta(\mu i, \mu j)}$	Low
Silhouette Co-efficient	$s_i = \frac{\mu_{out}^{min}(x_i) - \mu_{in}(x_i)}{\max\{\mu_{out}^{min}(x_i) - \mu_{in}(x_i)\}}$	High
Normalized Hubert Statistic	$\Gamma_n = \frac{\sum_{i=1}^{N-1}\sum_{j=i+1}^{N}(\mathbf{X}(i,j) - \mu_X).(\mathbf{Y}(i,j) - \mu_Y)}{\sqrt{\sum_{i=1}^{N-1}\sum_{j=i+1}^{N}(\mathbf{X}(i,j) - \mu_X)^2 . \sum_{i=1}^{N-1}\sum_{j=i+1}^{N}(\mathbf{Y}(i,j) - \mu_Y)^2}}$	High
Relative Inter-connectivity	$RI = \frac{1}{\binom{f}{2}}\sum_{i=1}^{f-1}\sum_{j>i}\frac{W(C_i, C_j)}{\frac{W(C_i, C_i) + W(C_j, C_j)}{2}}$	High

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

randomly. Table 3 shows the values of all the measures for generated datasets. The results clearly indicate that the values of these measures deteriorate with increase in distortion of spatial locality. Thus, they are suitable for our evaluation. Similar results are obtained for other datasets with higher number of compute nodes as well.

TABLE 3. VALIDATION OF QUALITY MEASURES

Dataset	BetaCV	Mod	Norm. Cut	DB	Norm. Hubt.	RelIntC	SIL
SC1M2D	0.175	-0.208	3.886	0.078	0.562	11.424	0.987
SC1M2D_10%_man	0.303	-0.181	3.807	0.177	0.467	6.588	0.943
SC1M2D_20%_man	0.414	-0.159	3.741	0.246	0.399	4.828	0.904
SC1M2D_30%_man	0.507	-0.142	3.688	0.307	0.346	3.943	0.871
SC1M2D_rand	0.820	-0.091	3.519	0.7346	0.112	2.438	0.743

In our experiments, we have observed that all the measures shown in Table 3 are behaving consistently with change in spatial locality. Hence in our subsequent presentation, we present our results with only two measures – BetaCV and Silhouette Co-efficient. The BetaCV measure is the ratio of the mean intra-cluster distance to the mean inter-cluster distance. The smaller the BetaCV ratio, the better the clustering, as it indicates that intra-cluster distances are on an average smaller than inter-cluster distances. The silhouette coefficient is a measure of both cohesion and separation of clusters, and is based on the difference between the average distance to points in the closest cluster and to points in the same cluster. For each point x_i we calculate its silhouette coefficient s_i as explained in Table 1. Here $\mu_{in}(x_i)$ is the mean distance from x_i to points in its own cluster y_i; and $\mu_{out}^{min}(x_i)$ is the mean of distances from x_i to points in the closest cluster. The silhoutte co-efficient of the entire clustering is the average of s_i of all points. The value of silhouette coefficeint lies in the interval [−1,+1]. A value close to +1 indicates that points are much closer to points in their own cluster and are far from other clusters, this indicating good clustering.

Quality Evaluation. Table 4 presents the quality measures of data distribution with DD-RTREE when compared with random distribution and distribution using SD-RTREE for various synthetic and real datasets. For synthetic random and synthetic normal datasets, the results show that the measures have always been consistently better for DD-RTREE than random distribution and SD-RTREE for different number of compute nodes used in the cluster. The quality of distribution for synthetic cluster dataset however, is slightly lower than that of SD-RTREE. This is because of difference in the re-distribution strategies used. Since the dataset has fully disjoint clusters, the k-NN based re-distribution strategy of SD-RTREE works better than the overlap based re-distribution strategy of DD-RTREE. The quality of distribution using DD-RTREE for all real datasets is found to be better than that of SD-RTREE. This shows that DD-RTREE is efficient in handling skewed datasets. For SFONT1M11D and MPAGD2.8M9D, DD-RTREE performs much better than others. This is because at high dimensional space, overlap based re-distribution of DD-RTREE performs considerably better than k-NN based re-distribution of SD-RTREE.

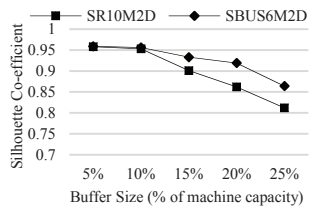


Figure 8. Silhouette Co-efficient with increase in buffer size

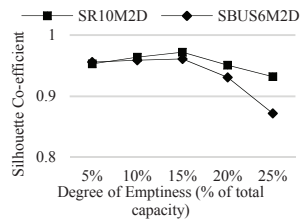


Figure 9. Silhouette Co-efficient with increase in degree of emptiness

Quality Analysis on Varying Factors. We analyze the quality of DD-RTREE with variation in buffer size and the degree of emptiness in the tree for 16 nodes in the cluster for SR10M2D and SBUS6M2D datasets. Results presented in Fig. 8 show that the quality of distribution deteriorates with increase in buffer size beyond 10% of machine capacity in both the cases. This is because large buffer size leads to infrequent flushes into MR-TREE. It also results in very large number of points being shifted in re-distributions. Optimal quality is observed for buffer sizes between 5 and 10% of machine capacity. Similarly, results presented in Fig. 9 show that the quality of distribution initially improves and then deteriorates when we increase the degree of emptiness in the compute nodes. This is because high degree of emptiness leads to data being distributed in a skewed manner.

We also compare the load balancing achieved for DD-RTREE and SD-RTREE and results are presented in Table 5 for SR10M2D dataset for n=16, when degree of emptiness is 5%. The results show that DD-RTREE achieves better load balance than SD-RTREE. Similar results were obtained for other datasets as well.

TABLE 4. DATA DISTRIBUTION QUALITY FOR VARYING NUMBER OF COMPUTE NODES FOR VARIOUS DATASETS

		n=16			n=32			n=64			n=128			n=256		
		rand	SDR	DDR	rand	SDR	DDR	rand	SDR	DDR	rand	SDR	DDR	rand	SDR	DDR
SR500M2D	BCV	0.715	0.526	0.439	0.632	0.518	0.426	0.698	0.516	0.428	0.702	0.511	0.403	0.697	0.506	0.411
	SIL	0.614	0.912	0.953	0.632	0.908	0.955	0.697	0.922	0.964	0.712	0.911	0.963	0.706	0.917	0.965
SN100M2D	BCV	0.765	0.699	0.698	0.712	0.624	0.614	0.703	0.617	0.598	0.71	0.599	0.572	0.685	0.548	0.516
	SIL	0.487	0.796	0.844	0.498	0.821	0.869	0.436	0.788	0.876	0.513	0.842	0.912	0.559	0.849	0.915
SC100M2D	BCV	0.833	0.411	0.451	0.795	0.396	0.432	0.768	0.347	0.386	0.778	0.301	0.941	0.724	0.282	0.304
	SIL	0.648	0.921	0.854	0.627	0.923	0.867	0.633	0.926	0.892	0.701	0.929	0.915	0.693	0.937	0.924
SFONT1M11D	BCV	0.514	0.398	0.248	0.534	0.375	0.244	0.527	0.368	0.214	0.486	0.347	0.196	0.447	0.329	0.204
	SIL	0.247	0.726	0.894	0.164	0.763	0.905	0.187	0.773	0.924	0.168	0.798	0.937	0.199	0.812	0.94
MPAHALO2.8M9D	BCV	0.628	0.583	0.473	0.604	0.562	0.436	0.593	0.518	0.401	0.562	0.501	0.386	0.579	0.483	0.372
	SIL	0.363	0.674	0.836	0.381	0.693	0.853	0.394	0.725	0.875	0.427	0.783	0.901	0.452	0.812	0.894
FOF113M3D	BCV	0.864	0.635	0.447	0.822	0.622	0.432	0.812	0.593	0.458	0.794	0.605	0.405	0.807	0.586	0.415
	SIL	0.294	0.764	0.889	0.386	0.793	0.892	0.357	0.813	0.914	0.429	0.818	0.927	0.414	0.827	0.942
MPAGD56M3D	BCV	0.62	0.455	0.412	0.637	0.441	0.399	0.587	0.428	0.394	0.641	0.427	0.367	0.623	0.428	0.338
	SIL	0.749	0.901	0.914	0.726	0.92	0.921	0.738	0.914	0.908	0.722	0.908	0.917	0.685	0.917	0.921
SBUS6M2D	BCV	0.512	0.458	0.345	0.539	0.469	0.314	0.521	0.431	0.324	0.566	0.413	0.305	0.547	0.407	0.298
	SIL	0.802	0.947	0.956	0.765	0.952	0.974	0.744	0.962	0.966	0.798	0.967	0.982	0.783	0.963	0.971

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

TABLE 5. LOAD BALANCING FOR SR10M2D DATASET

Data Structure	Range of number of points in machines
SD-RTREE	46,216 – 62,500
DD-RTREE	56,394 – 60,341

B. *Efficiency Evaluation*

We evaluate DD-RTREE for execution time and MPI messages required for data distribution.

Construction of DD-RTREE. We measure construction time and number of MPI messages required for DD-RTREE and SD-RTREE on SR10M2D and SBUS6M2D datasets for 16 and 32 nodes. The results presented in Table 6 show that the execution time and the MPI messages required for DD-RTREE is less with respect to SD-RTREE. This is mainly attributed to buffering technique used to defer insertions and inserting them in bulk rather than point by point. This is also due to reduction in number of re-distributions for DD-RTREE.

TABLE 6. CONSTRUCTION TIME OF DD-RTREE VS SD-RTREE

Dataset	SR10M2D			SBUS6M2D		
No of Nodes	Data Structure	Construction Time	MPI Messages (approx..)	Data Structure	Construction Time	MPI Messages (approx..)
n=16	SD-RTREE	1787 sec.	25.3 M	SD-RTREE	1607 sec.	18.4 M
	DD-RTREE	1295 sec.	1.4 M	DD-RTREE	1064 sec.	1.2 M
n=32	SD-RTREE	1839 sec.	26.4 M	SD-RTREE	1672 sec.	19.6 M
	DD-RTREE	1363 sec.	1.6 M	DD-RTREE	1114 sec.	1.3 M

We have also measured the construction time of DD-RTREE and number of MPI messages required, with variation in buffer size (% of machine capacity) for SR10M2D and SBUS6M2D datasets on 32 nodes. The results presented in Table 7 indicate that the number of MPI messages decrease as the buffer size increases. This is because when buffer size is small, the buffer is flushed very frequently and re-distribution routine is executed more number of times. However, we can see from Fig. 8, that quality of distribution is good when the buffer size is small. Therefore, we have taken buffer size to be 10% in all our experimentation.

TABLE 7. CONSTRUCTION TIME OF DD-RTREE WITH VARIATION IN BUFFER SIZE

		5%	10%	15%	20%	25%
SR10M2D	Construction Time	1462 sec.	1363 sec.	1284 sec.	1106 sec.	1085 sec.
	MPI messages	1.8 M	1.6 M	1.2 M	0.9 M	0.8 M
SBUS6M2D	Construction Time	1267 sec.	1114 sec.	1068 sec.	1027 sec.	992 sec.
	MPI messages	1.5 M	1.3 M	0.8 M	0.6 M	0.5 M

Performance of k-d-tree distribution using DD-RTREE. We measure the total number of data points shifted from one machine to other machine in the naïve k-d-tree data distribution and k-d-tree distribution using DD-RTREE for SR100M2D and MPAGD56M dataset over 32 machines. Table 8 shows that the number of points shifted from one machine to other and the execution time for DD-RTREE is less than the naïve approach for both datasets.

TABLE 8. PERFORMANCE OF K-D-TREE LIKE DISTRIBUTION

DataSet	SR100M2D		MPAGD56M	
	Brute-force	DD-RTREE	Brute-force	DD-RTREE
Execution Time	4573 sec	3295 sec	2764 sec	1834 sec
Number of points shifted	278 M (approx.)	164 M (approx.)	210 M (approx..)	130 M (approx..)

Performance of Queries. We measure the average number of machines visited per query, average number of MPI messages and average execution time, for ε-neighborhood and k-NN queries when executed over DD-RTREE and SD-RTREE for SR100M2D and MPAGD56M3D datasets for 32 nodes. We have used 10% sample of the dataset as querying points and have computed averages. We choose ε=0.01 for SR100M2D dataset and ε=0.006 for MPAGD56M3D dataset for executing neighborhood queries. We choose k=20 for all k-NN queries. The results presented in Table 9 clearly indicate that all these parameters are better for DD-RTREE, proving its better spatial locality. We also observed that query performance of DD-RTREE is consistently maintained with variation in ε for neighborhood queries and variation in k for k-NN queries when compared to SD-RTREE. Due to limited space, we don't present those results.

TABLE 9. QUERYING PERFORMANCE OF DD-RTREE AND SD-RTREE

		SR100M2D			MPAGD56M		
		Average MPI Messages	Average Execution Time	Average number of Machines Visited	Average MPI Messages	Average Execution Time	Average number of Machines Visited
DD-RTREE	Nbh Query	3.18	0.051 sec.	1.59	2.90	0.043 sec.	1.45
	k-NN query	3.17	0.074 sec.	2.17	2.96	0.058 sec.	1.96
SD-RTREE	Nbh Query	3.44	0.069 sec.	1.72	3.26	0.056 sec.	1.63
	k-NN query	3.38	0.086 sec.	2.38	3.24	0.072 sec.	2.24

Performance of Distributed DBSCAN. We perform simple version of distributed DBSCAN over SD-RTREE, DD-RTREE and distributed k-d-tree distribution using DD-RTREE, to compare their performance over MPAGD16M3D dataset over 32 machines. The ε was chosen to 0.01 and Min_Pts was chosen to be 5. Table 10 presents the summary of its execution. DBSCAN follows all four steps of a distributed algorithm explained in section 1. In step 1, we distributed data using suitable method. In this DD-RTREE and k-d-tree distribution using DD-RTREE take less time when compared with SD-RTREE. In step 2, for every machine, we retrieve data points from other machines in the cluster which lie within ε-extended boundary of the current machine. In this step, the number of MPI messages required remains same in all cases. However, the execution time for DD-RTREE is lesser because the number of extra data points fetched from other machines is less. We can also observe that in k-d-tree based distribution using DD-RTREE, the number of points retrieved from other machines is very small. This is because, the machines are fully disjoint in their distribution. Step 3 involves execution of local DBSCAN at each machine. In this step, the time require for local DBSCAN is less for DD-RTREE and k-d-tree based distribution, because of lesser number of extra points at each node helps in reducing time for neighborhood queries. In step 4, we merge the results of all local DBSCAN to give the required global clustering. In this step, the time required for merging is almost the same for DD-RTREE and k-d-tree distribution using DD-RTREE and better than SD-RTREE. This shows that effective distribution of data enables reduction in communication complexity and improvement in the execution time of parallel DBSCAN algorithm.

TABLE 10. DBSCAN USING DD-RTREE AND SD-RTREE

	SD-RTREE	DD-RTREE	DISTRIBUTED K-D-TREE USING DD-RTREE
	Execution Time		
Step 1	1948 sec.	1426 sec	1639 sec.
Step 2	481 sec.	304 sec.	214 sec.
Step 3	1671 sec.	1428 sec.	919 sec.
Step 4	150 sec.	139 sec.	131 sec.
Total Execution Time	**4250 sec.**	**3297 sec.**	**2903 sec.**
	Number of data points retrieved from other machines		
Step 2	1.92 M	1.28 M	0.72 M

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.

V. Conclusions & Future Work

This paper proposes DD-RTREE, which is a dynamic distributed data structure based on R-tree. DD-RTREE preserves spatial locality in its distribution, achieves good load balancing, exhibits less communication overhead in querying and construction, and improves the performance of parallel spatial data mining algorithms. DD-RTREE also supports efficient execution of ε-neighborhood and k-NN queries. It also enables efficient k-d-tree distribution over cluster nodes for very large datasets. The quality and efficiency evaluation together establish the superiority of DD-RTREE with respect to SD-RTREE and random distribution.

DD-RTREE can be used to design highly efficient distributed framework for mining data streams. Also, the DD-RTREE strategy can very well be applied on other variants of R-tree as well.

Acknowledgment

This work has been partially supported by a research grant from Dept. of Electronics and IT (DeiTy), Govt. of India.

References

[1] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *Second International Conference on Knowledge Discovery and Data Mining*, pp. 226–231, 1996.

[2] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "OPTICS : Ordering Points To Identify the Clustering Structure," *ACM SIGMOD Record*, vol. 28, no. 2, pp. 49–60, Jun. 1999.

[3] R. Sibson, "SLINK: An optimally efficient algorithm for the single-link cluster method," *The Computer Journal*, vol. 16, no. 1, pp. 30–34, Jan. 1973.

[4] T. Pang-Ning, M. Steinbach, and V. Kumar, *Introduction to data mining*, 1st Editio. Addison-Wesley Longman Publishing Co. Boston MA, 2006.

[5] P. Goyal, S. Kumari, D. Kumar, S. Balasubramaniam, N. Goyal, S. Islam, and J. S. Challa, "Parallelizing OPTICS for Commodity Clusters," in *Proceedings of the 2015 International Conference on Distributed Computing and Networking - ICDCN '15*, 2015, pp. 1–10.

[6] W. Hendrix, M. M. Ali Patwary, A. Agrawal, W. K. Liao, and A. Choudhary, "Parallel hierarchical clustering on shared memory platforms," in *2012 19th International Conference on High Performance Computing, HiPC 2012*, 2012, pp. 1–9.

[7] M. M. A. Patwary, D. Palsetia, A. Agrawal, W. K. Liao, F. Manne, and A. Choudhary, "A new scalable parallel DBSCAN algorithm using the disjoint-set data structure," in *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*, 2012, pp. 1–11.

[8] M. M. A. Patwary, S. Byna, N. R. Satish, N. Sundaram, Z. Lukic, V. Roytershteyn, M. J. Anderson, Y. Yao, Prabhat, and P. Dubey, "BD-CATS: big data clustering at trillion particle scale.," in *SC15: International Conference for High Performance Computing, Networking, Storage and Analysis'15*, 2015, pp. 1–12.

[9] B. Welton, E. Samanas, and B. P. Miller, "Mr. Scan: Extreme Scale Density-based Clustering Using a Tree-based Network of GPGPU Nodes," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis on - SC '13*, 2013, pp. 1–11.

[10] I. Kamel, C. Faloutsos, I. Kamel, and C. Faloutsos, "Parallel R-trees," in *SIGMOD '92 Proceedings of the 1992 ACM SIGMOD international conference on Management of data*, 1992, pp. 195–204.

[11] E. G. Hoel and H. Samet, "Data-Parallel R-Tree Algorithms," in *1993 International Conference on Parallel Processing - ICPP'93*, 1993, pp. 49–52.

[12] T. Johnson and A. Colbrook, "A distributed data-balanced dictionary based on the B-link tree," in *Proceedings Sixth International Parallel Processing Symposium*, 1992, pp. 319–324.

[13] K. Brigitte and W. Peter, "Distributing a Search Tree Among a Growing Number of Processors," in *SIGMOD '94 Proceedings of the 1994 ACM SIGMOD international conference on Management of data*, 1994, pp. 265–276.

[14] B. Schnitzer and S. T. Leutenegger, "Master-client R-trees: a new parallel R-tree architecture," in *Proceedings of Eleventh International Conference on Scientific and Statistical Database Management (SSDBM'99)*, 1999, pp. 68–77.

[15] S. Lai, F. Zhu, and Y. Sun, "A Design of Parallel R-tree on Cluster of Workstations," in *International Workshop Databases in Networked Information Systems*, 2000, pp. 119–133.

[16] C. Du Mouza, W. Litwin, and P. Rigaux, "Large-scale indexing of spatial data in distributed repositories: The SD-Rtree," *VLDB Journal*, vol. 18, no. 4, pp. 933–958, 2009.

[17] T. H. Cormen, C. Stein, R. L. Rivest, and C. E. Leiserson, "Introduction to Algorithms," Jul. 2001.

[18] A. Guttman, "R-Trees. a Dynamic Index Structure for Spatial Searching," in *In Proceedings of the 1984 ACM SIGMOD international conference on Management of data (SIGMOD '84)*, 1984, pp. 47–57.

[19] E. N. Adriano Di Pasquale, "Scalable Distributed Data Structures: A Survey," in *3rd International Workshop on Distributed Data and Structures (WDAS'00)*, 2000, pp. 87–111.

[20] T. P. Hayes, J. Saia, and A. Trehan, "The Forgiving Graph: a distributed data structure for low stretch under adversarial attack," *Distributed Computing*, vol. 25, no. 4, pp. 261–278, Feb. 2012.

[21] M. T. Goodrich, M. J. Nelson, and J. Z. Sun, "The Rainbow Skip Graph: A Fault-Tolerant Constant-Degree Distributed Data Structure," in *Proceedings of the seventeenth annual ACM-SIAM symposium on Discrete algorithm - SODA '06*, 2006, pp. 384–393.

[22] A. Akdogan, U. Demiryurek, F. Banaei-Kashani, and C. Shahabi, "Voronoi-Based Geospatial Query Processing with MapReduce," in *2010 IEEE Second International Conference on Cloud Computing Technology and Science*, 2010, pp. 9–16.

[23] G. R. Hjaltason and H. Samet, "Distance browsing in spatial databases," *ACM Transactions on Database Systems*, vol. 24, no. 2, pp. 265–318, 1999.

[24] G. M. Adelson-Velskii and E. M. Landis, "An algorithm for the organization of information," *Doklady Akademii Nauk SSSR*, vol. 146, pp. 263–266, 1962.

[25] V. Springel, S. D. M. White, A. Jenkins, C. S. Frenk, N. Yoshida, L. Gao, J. Navarro, R. Thacker, D. Croton, J. Helly, J. A. Peacock, S. Cole, P. Thomas, H. Couchman, A. Evrard, J. Colberg, and F. Pearce, "Simulations of the formation, evolution and clustering of galaxies and quasars.," *Nature*, vol. 435, no. 7042, pp. 629–36, Jun. 2005.

[26] "SUVN Trace Data." [Online]. Available: http://wirelesslab.sjtu.edu.cn/. [Accessed: 17-Sep-2015].

[27] M. J. Zaki and W. Meira, *Data Mining and Analysis: fundamental concepts and algorithms.* 2014.

[28] G. Karypis, E. H. Han, and V. Kumar, "Chameleon: Hierarchical clustering using dynamic modeling," *Computer*, vol. 32, no. 8, pp. 68–75, 2002.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on March 10,2026 at 04:23:37 UTC from IEEE Xplore. Restrictions apply.